# Relational Algebra

UNIT-3(b)

# Relational Algebra in DBMS

- Relational Algebra came in 1970 and was given by Edgar F. Codd (Father of DBMS).
- It is also known as Procedural Query Language(PQL) as in PQL.
- A programmer/user has to mention two things, **"What to Do"** and **"How to Do"**.
- Queries in relational algebra are performed using operators.

# Why Use Relational Algebra?

- It forms the theoretical foundation for SQL.
- It allows you to express queries precisely and step-by-step.
- It ensures efficient query optimization in RDBMS.
- **Relational Algebra** concepts are practically implemented in database systems, especially in **SQL**
- **Relational Algebra** itself is a theoretical framework, modern RDBMS

# How Relational Algebra Relates to SQL

| Relational Algebra | SQL Equivalent |
|---|---|
| Selection (σ) | `WHERE` clause |
| Projection (π) | `SELECT` statement |
| Union ( ∪ ) | `UNION` |
| Intersection ( ∩ ) | `INTERSECT` |
| Difference ( - ) | `EXCEPT` (or `MINUS` in Oracle) |
| Cartesian Product ( × ) | Implicit when no `JOIN` condition is specified |
| Join ( ⋈ ) | `INNER JOIN`, `LEFT JOIN`, etc. |
| Rename (ρ) | `AS` keyword for aliasing |

- **Example in Practice**
- **Relational Algebra Expression:**
  $\pi$ Name ($\sigma$ Age > 20 (Students))

- SQL Equivalent:

```
SELECT Name
FROM Students
WHERE Age > 20;
```

# Types of Relational Operations in DBMS

- **Basic Operations**
- **Derived Operations**

# Types of Relational Operations

- **Basic Operations**
  - Select ($\sigma$)
  - Project ($\pi$)
  - Union ($\cup$)
  - Set Difference ($-$)
  - Cartisian Product ($\times$)
  - Rename ($\rho$)
- **Derived Operations**
  - Division ($\div$)
  - Intersection ($\cap$)
  - Join
    - Inner
      - Theta ($\theta$)
      - Equi
      - Natural ($\bowtie$)
    - Outer
      - Left Outer ($\bowtie$)
      - Right Outer ($\bowtie$)
      - Full Outer ($\bowtie$)

# Basic Operations

Six fundamental operations are mentioned below.

## 1. Select (σ)

- Select operation is done by Selection Operator which is represented by "sigma"(σ).
- It is used to retrieve tuples(rows) from the table where the given condition is satisfied.
- It is a **unary operator** means it requires only one operand.

**Example** Notation : σ p(R)

Where σ is used to represent SELECTION

R is used to represent RELATION

p is the logic formula

**Suppose we want the row(s) from STUDENT Relation where "AGE" is 20**

- σ AGE=20 (STUDENT)

STUDENT

| ROLL | NAME | AGE |
|---|---|---|
| 1 | Aman | 20 |
| 2 | Atul | 18 |
| 3 | Baljeet | 19 |
| 4 | Harsh | 20 |
| 5 | Prateek | 21 |
| 6 | Prateek | 23 |

This will return the following output:

| ROLL | NAME | AGE |
|---|---|---|
| 1 | Aman | 20 |
| 4 | Harsh | 20 |

# Basic Operations

## 2. Project (∏)

- Project operation is done by Projection Operator which is represented by "pi"(∏).
- It is used to retrieve certain attributes(columns) from the table.
- It is also known as vertical partitioning as it separates the table vertically.
- It is also a **unary operator**.

Notation : ∏ a(r)

Where ∏ is used to represent PROJECTION
r is used to represent RELATION
a is the attribute list

Suppose we want the names of all students from STUDENT Relation.

**∏ NAME(STUDENT)**

- This will return the following output:

| NAME |
| --- |
| Aman |
| Atul |
| Baljeet |
| Harsh |
| Prateek |

- For multiple attributes, we can separate them using a ",".
- ∏ ROLL,NAME(STUDENT)

| ROLL | NAME |
| --- | --- |
| 1 | Aman |
| 2 | Atul |
| 3 | Baljeet |
| 4 | Harsh |
| 5 | Prateek |
| 6 | Prateek |

- As you can see from the above output it **eliminates duplicates**.

# Basic Operations

## 3. Union ( ∪ )

- Union operation is done by Union Operator which is represented by "union"( ∪ )
- **it selects all tuples from both relations but with the exception that for the union of two relations/tables both relations must have the same set of Attributes**.
- It is a **binary operator** as it requires two operands.
- Notation: R ∪ S

Where R is the first relation

S is the second relation

- Suppose we want all the names from STUDENT and EMPLOYEE relation.
- **∏ NAME(STUDENT) ∪ ∏ NAME(EMPLOYEE)**

STUDENT

| ROLL | NAME |
|---|---|
| 1 | Aman |
| 2 | Atul |
| 3 | Baljeet |
| 4 | Harsh |
| 5 | Prateek |
| 6 | Prateek |

| NAME |
|---|
| Aman |
| Anant |
| Ashish |
| Atul |
| Baljeet |
| Harsh |
| Pranav |
| Prateek |

| EMPLOYEE_NO | NAME | AGE |
|---|---|---|
| E-1 | Anant | 20 |
| E-2 | Ashish | 23 |
| E-3 | Baljeet | 25 |
| E-4 | Harsh | 20 |
| E-5 | Pranav | 22 |

# Basic Operations

## 4. Set Difference (-)

- Set Difference as its name indicates is the difference between two relations (R-S).
- It is denoted by a "Hyphen"(-) and it returns all the tuples(rows) which are in relation R but not in relation S.
- It is a **binary operator** as it requires two operands.
- **Notation : R – S**

Where R is the first relation

S is the second relation

- ∏ NAME(STUDENT) - ∏ NAME(EMPLOYEE)

This will give us the following output:

| NAME |
| --- |
| Aman |
| Atul |
| Prateek |

# Basic Operations

## 5. Cartesian product (X)

- Cartesian product is denoted by the "X" symbol.
- Cartesian product will combine every tuple(row) from R with all the tuples from S.
- **Notation: R X S**
- Where R is the first relation

S is the second relation

- the notation it is also a **binary operator**.
- **STUDENT X EMPLOYEE**

STUDENT

| ROLL | NAME | AGE |
|---|---|---|
| 1 | Aman | 20 |
| 2 | Atul | 18 |
| 3 | Baljeet | 19 |
| 4 | Harsh | 20 |
| 5 | Prateek | 21 |
| 6 | Prateek | 23 |

EMPLOYEE

| EMPLOYEE_NO | NAME | AGE |
|---|---|---|
| E-1 | Anant | 20 |
| E-2 | Ashish | 23 |
| E-3 | Baljeet | 25 |
| E-4 | Harsh | 20 |
| E-5 | Pranav | 22 |

| ROLL | NAME | AGE | EMPLOYEE_NO | NAME | AGE |
|---|---|---|---|---|---|
| 1 | Aman | 20 | E-1 | Anant | 20 |
| 1 | Aman | 20 | E-2 | Ashish | 23 |
| 1 | Aman | 20 | E-3 | Baljeet | 25 |
| 1 | Aman | 20 | E-4 | Harsh | 20 |
| 1 | Aman | 20 | E-5 | Pranav | 22 |
| 2 | Atul | 18 | E-1 | Anant | 20 |
| 2 | Atul | 18 | E-2 | Ashish | 23 |
| 2 | Atul | 18 | E-3 | Baljeet | 25 |
| 2 | Atul | 18 | E-4 | Harsh | 20 |
| 2 | Atul | 18 | E-5 | Pranav | 22 |

# Basic Operations

## 6. Rename (ρ)

- Rename operation is denoted by "Rho"(ρ).
- As its name suggests it is used to rename the output relation.
- Rename operator too is a **binary operator**.
- **Notation: ρ(R,S)**
  Where R is the new relation name
  S is the old relation name

Suppose we are fetching the names of students from STUDENT relation. We would like to rename this relation as STUDENT_NAME.

**ρ(STUDENT_NAME,∏ NAME(STUDENT))**

- STUDENT_NAME

| NAME |
| --- |
| Aman |
| Atul |
| Baljeet |
| Harsh |
| Prateek |

# Quick Revision

- Select (σ) is used to retrieve tuples(rows) based on certain conditions.
- Project (∏) is used to retrieve attributes(columns) from the relation.
- Union ( ∪ ) is used to retrieve all the tuples from two relations.
- Set Difference (-) is used to retrieve the tuples which are present in R but not in S(R-S).
- Cartesian product (X) is used to combine each tuple from the first relation with each tuple from the second relation.
- Rename (ρ) is used to rename the output relation.

# Types of Relational Operations

- **Basic Operations**
  - Select ($\sigma$)
  - Project ($\pi$)
  - Union ($\cup$)
  - Set Difference ($-$)
  - Cartisian Product ($\times$)
  - Rename ($\rho$)
- **Derived Operations**
  - Division ($\div$)
  - Intersection ($\cap$)
  - Join
    - Inner
      - Theta ($\theta$)
      - Equi
      - Natural ($\bowtie$)
    - Outer
      - Left Outer ($\bowtie$)
      - Right Outer ($\bowtie$)
      - Full Outer ($\bowtie$)

# Derived Operations

- Also known as extended operations
- These operations can be derived from basic operations and hence named Derived Operations
- These include three operations:
  - Join Operations
  - Intersection operations
  - Division operations.

# Join Operations

- **Join Operation in DBMS** are binary operations that allow us to combine two or more relations.
- They are further classified into two types:
  - Inner Join
  - Outer Join.

  In SQL, **INNER JOIN** and **OUTER JOIN** are used to combine rows from two or more tables based on a related column between them.
- An **INNER JOIN** returns only the rows that have matching values in both tables.
- An **OUTER JOIN** returns all the records from one table and the matching records from the other table. If there's no match, NULL is returned for the missing values.

# Theta Join (θ)

- Theta Join combines two relations using a condition.
- This condition is represented by the symbol "theta"(θ).
- Here conditions can be inequality conditions such as >,<,>=,<=, etc.

  **Notation : $R \bowtie_\theta S$**

  Where R is the first relation

  S is the second relation

# Equi Join

- Equi Join is a special case of theta join where the condition can only contain

  **equality(=)** comparisons.
- A non-equijoin is the inverse of an equi join, which occurs when you join on a condition other than "=".

# Natural Join (⋈)

- **A comparison operator is not used in a natural join.**
- A Natural Join can be performed only if two relations share at least **one common attribute**. Furthermore, the attributes **must share the same name and domain**.
- Natural join operates on matching attributes where the values of the attributes in both relations are the same and **remove the duplicate** ones.
- **Preferably Natural Join is performed on the foreign key.**
- **Notation : R ⋈ S**
  Where R is the first relation
  S is the second relation

- Let's say we want to join EMPLOYEE and DEPARTMENT relation with E_NO as a common attribute.

EMPLOYEE

| E_NO | E_NAME | CITY | EXPERIENCE |
|---|---|---|---|
| E-1 | Ram | Delhi | 04 |
| E-2 | Varun | Chandigarh | 09 |
| E-3 | Ravi | Noida | 03 |
| E-4 | Amit | Bangalore | 07 |

DEPARTMENT

| D_NO | D_NAME | E_NO | MIN_EXPERIENCE |
|---|---|---|---|
| D-1 | HR | E-1 | 03 |
| D-2 | IT | E-2 | 05 |
| D-3 | Marketing | E-3 | 02 |

| E_NO | E_NAME | CITY | EXPERIENCE | D_NO | D_NAME | MIN_EXPERIENCE |
|---|---|---|---|---|---|---|
| E-1 | Ram | Delhi | 04 | D-1 | HR | 03 |
| E-2 | Varun | Chandigarh | 09 | D-2 | IT | 05 |
| E-3 | Ravi | Noida | 03 | D-3 | Marketing | 02 |

# Intersection (∩)

- Intersection operation is done by Intersection Operator which is represented by "intersection"(∩).
- It selects all the tuples which are present in both relations.
- It is a **binary operator** as it requires two operands. Also, it **eliminates duplicates**.

  **Notation : R ∩ S**

  Where R is the first relation

  S is the second relation

- Suppose we want the names which are present in STUDENT as well as in EMPLOYEE relation, Relations we used in Basic Operations.
- $\prod$ NAME(STUDENT) $\cap$ $\prod$ NAME(EMPLOYEE)

STUDENT

| ROLL | NAME | |
|---|---|---|
| 1 | Aman | |
| 2 | Atul | |
| 3 | Baljeet | |
| 4 | Harsh | 20 |
| 5 | Prateek | 21 |
| 6 | Prateek | 23 |

| NAME |
|---|
| Baljeet |
| Harsh |

EMPLOYEE

| PLOYEE_NO | NAME | AGE |
|---|---|---|
| E-1 | Anant | 20 |
| E-2 | Ashish | 23 |
| E-3 | Baljeet | 25 |
| E-4 | Harsh | 20 |
| E-5 | Pranav | 22 |

# Division (÷)

- Division Operation is represented by "division"(÷ or /) operator
- It is used in queries that involve keywords **"every"**, **"all"**, etc.
  **Notation : R(X,Y)/S(Y)**
  Here,
  R is the first relation from which data is retrieved.
  S is the second relation that will help to retrieve the data.
  X and Y are the attributes/columns present in relation

ENROLLED

| STUDENT_ID | COURSE_ID |
|---|---|
| Student_1 | DBMS |
| Student_2 | DBMS |
| Student_1 | OS |
| Student_3 | OS |

COURSE

| COURSE_ID |
|---|
| DBMS |
| OS |

**ENROLLED(STUDENT_ID, COURSE_ID)/COURSE(COURSE_ID)**

| STUDENT_ID |
|---|
| Student_1 |

# Conclusion

- Relational Algebra in DBMS is a theoretical model which is the fundamental block for SQL. It comprises different mathematics operations.
- Operations are divided into two main categories: Basic and Derived.
- **Basic consists of six Operations: SELECT, PROJECT, UNION, SET DIFFERENCE, CARTESIAN PRODUCT, RENAME**.
- **Derived Consist of three Operations: JOINS, INTERSECTION, DIVISION**.